

《LeetCode零基础指南》(第九讲) 简单递归

文章目录

- 零、了解网站
 - 1、输入输出
 - 2、刷题步骤
 - 3、尝试编码
 - 4、调试提交
- 一、概念定义
 - 1、递归的含义
 - 2、递归调用阶乘
 - 1) 实现一个函数
 - 2) 递归出口
 - 3) 递推关系
 - 3、为什么叫递归
- 二、题目分析
 - 1、阶乘尾后零
 - 2、将数字变成 0 的操作次数
 - 3、完全二叉树的节点个数
 - 4、开幕式
- 三、推荐学习
- 四、课后习题

零、了解网站

1、输入输出

对于算法而言，就是给定一些输入，得到一些输出，在一般的在线评测系统中，我们需要自己手写输入输出函数（例如 C 语言中的 `scanf` 和 `printf`），而在 **LeetCode** 这个平台，只需要实现它提供的函数即可。函数的传入参数就代表了算法的输入，而函数的返回值则代表了算法的输出。

2、刷题步骤

找到一道题，以 **两整数之和** 为例，如何把这道题通过呢？如下图所示：

The screenshot shows the LeetCode interface for the problem "371. 两整数之和" (Two Sum). The problem description states: "给你两个整数 `a` 和 `b`，不使用运算符 `+` 和 `-`，计算并返回两整数之和。" (Given two integers `a` and `b`, return their sum without using the operators `+` and `-`). The examples show: "示例 1: 输入: `a = 1, b = 2` 输出: `3`" and "示例 2: 输入: `a = 2, b = 3` 输出: `5`". The constraints are: "`-1000 <= a, b <= 1000`". The code editor shows a C function signature: `int getSum(int a, int b)`. The bottom of the page has buttons for "执行代码" (Run Code) and "提交" (Submit).

Annotations on the image:

- 1: Points to the problem description.
- 2: Points to the examples.
- 3: Points to the constraints.
- 4: Points to the code editor.
- 5: Points to the "执行代码" (Run Code) button.
- 6: Points to the "提交" (Submit) button.

- 第 1 步：阅读题目；
- 第 2 步：参考示例；
- 第 3 步：思考数据范围；
- 第 4 步：根据题意，实现函数的功能；
- 第 5 步：本地数据测试；
- 第 6 步：提交；

3、尝试编码

这个题目比较简单，就是求两个数的和，我们可以在编码区域尝试敲下这么一段代码。

```
1 int getSum(int a, int b){ // (1)
2     return a + b;        // (2)
3 }
```

- (1) 这里 `int` 是C/C++中的一种类型，代表整数，即 Integer，传入参数是两个整数；
- (2) 题目要求返回两个整数的和，我们用加法运算符实现两个整数的加法；

4、调试提交

力扣 学习 题库 讨论 竞赛 求职 商店

题目描述 评论 (548) 题解 (510) 提交记录

执行结果: 通过 显示详情

执行用时: 0 ms, 在所有 C 提交中击败了 100.00% 的用户

内存消耗: 5.5 MB, 在所有 C 提交中击败了 17.94% 的用户

通过测试用例: 25 / 25

炫耀一下:

写题解, 分享我的解题思路

提交结果	执行用时	内存消耗	提交时间	备注
通过	0 ms	5.5 MB	C	2021/11/22 22:33
通过	0 ms	5.2 MB	C	2021/11/22 13:21

C 智能模式 模拟面试

```
1 int getSum(int a, int b){
2     return a + b;
3 }
4
```

测试用例 代码执行结果 调试器

已完成 执行用时: 0 ms

输入: 45, 289

输出: 334

预期结果: 334

控制台 填入示例 如何创建一个测试用例?

执行代码 提交

- 第 1 步：实现加法，将值返回；
- 第 2 步：执行代码，进行测试数据的调试；
- 第 3 步：代码的实际输出；
- 第 4 步：期望的输出，如果两者符合，则代表这一组数据通过（但是不代表所有数据都能通过哦）；
- 第 5 步：尝试提交代码；
- 第 6 步：提交通过，撒花 🌸🌸🌸🌸🌸🌸；

这样，我们就大致知道了一个刷题的步骤了。
今天作为九日集训的最后一天，我们来学习一下作为程序新人来说，最为头疼的内容：递归！

一、概念定义

1、递归的含义

递归就是函数自己调自己。

递归只要记住三点内容：

- 1) 你要实现一个函数，这个函数会自己调用自己，并且每次调用，函数传参是不一样的；

- 2) 递归一定要有出口，即满足一定条件后需要 **return**，否则就可能出现 **死递归**（引起栈溢出）；
- 3) 根据递推式来补充你的递归调用内容；

2、递归调用阶乘

我们知道，阶乘即

$$n! = n * (n - 1) * (n - 2) * \dots * 3 * 2 * 1$$

光知道这个，我们是无法将它进行递归计算的，我们需要将它转换成递推式，如下：

$$n! = n * (n - 1)!$$

这时候，我们令 $f(n) = n!$ ，则有：

$$f(n) = n f(n - 1)$$

这就是阶乘的递推式。

然后，我们根据上面提到的三个点，进行如下套用：

1) 实现一个函数

这个函数叫 **JieCheng**，它的参数是一个整数，返回值也是一个整数，实现如下：

```
1 | int JieCheng(int n) {  
2 | }
```

2) 递归出口

当 n 等于 0 或 1 的时候， $n!$ 的值都为 1，所以递归出口如下：

```
1 | int JieCheng(int n) {  
2 |     if(n <= 1) {  
3 |         return 1;  
4 |     }  
5 | }
```

3) 递推关系

最后，我们将递推关系补充完毕，这样，一个递归函数就实现完毕了。

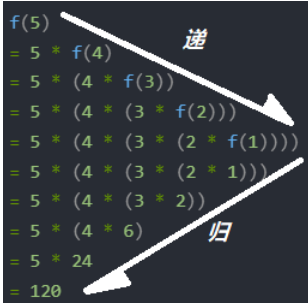
```
1 | int JieCheng(int n) {  
2 |     if(n <= 1) {  
3 |         return 1;  
4 |     }  
5 |     return n * JieCheng(n-1);  
6 | }
```

3、为什么叫递归

我们按照递推式，将递归调用展开如下：

```
1 | f(5)  
2 | = 5 * f(4)  
3 | = 5 * (4 * f(3))  
4 | = 5 * (4 * (3 * f(2)))  
5 | = 5 * (4 * (3 * (2 * f(1))))  
6 | = 5 * (4 * (3 * (2 * 1)))  
7 | = 5 * (4 * (3 * 2))  
8 | = 5 * (4 * 6)  
9 | = 5 * 24  
10 | = 120
```

这时候我们就会发现，它是两个过程：递推 和 回归（又叫回溯）。如下图所示：



二、题目分析

1、阶乘尾后零

给定一个整数 n ，返回 $n!$ 结果中尾随零的数量。提示 $n! = n * (n - 1) * (n - 2) * \dots * 3 * 2 * 1$

```
1 int trailingZeroes(int n){
2     if(n < 5) {
3         return 0; // (1)
4     }
5     return n / 5 + trailingZeroes(n/5); // (2)
6 }
```

- (1) 当 $n < 5$ 的时候，由于 $n!$ 中没有 10 这个因子，尾部自然是没有零的；
- (2) 于是，我们发现，问题本质就是求 10 的因子的数量，而在 $n!$ 中，2 的因子不会少于 5。所以其实试求 5 的因子数量。

求 5 的因子数量，我们可以把所有 5 的倍数（且非 25 的倍数）找出来，有 $\frac{n}{5}$ 个；所有 25 的倍数（且非 625 的倍数）找出来， $\frac{n}{25}$ 个；所有 125 的倍数找出来， $\frac{n}{625}$ 个；以此类推，所以，我们令 $f(n)$ 表示 n 的 5 因子数，它就等于：

$$f(n) = \frac{n}{5} + \frac{n}{25} + \frac{n}{125} + \dots$$

再将它转换成递推式，得到：

$$f(n) = \frac{n}{5} + f(\frac{n}{5})$$

然后就可以用递推式来套用递归求解了。

2、将数字变成 0 的操作次数

给你一个非负整数 num，请你返回将它变成 0 所需要的步数。如果当前数字是偶数，你需要把它除以 2；否则，减去 1。

```
1 int numberOfSteps(int num){
2     if(num == 0) {
3         return 0; // (1)
4     }
5     if(num % 2 == 1) {
6         return numberOfSteps(num-1)+1; // (2)
7     }else {
8         return numberOfSteps(num/2) + 1; // (3)
9     }
10 }
```

- (1) 递归出口；
- (2) 当 n 是奇数的情况，就是 $n - 1$ 的状态走一步过来的；
- (3) 当 n 为偶数的情况，就是 $n/2$ 的状态走一步过来的；

3、完全二叉树的节点个数

给你一棵 完全二叉树 的根节点 `root`，求出该树的节点个数。完全二叉树 的定义如下：在完全二叉树中，除了最底层节点可能没填满外，其余每层节点数都达到最大值，并且最下面一层的节点都集中在该层最左边的若干位置。若最底层为第 h 层，则该层包含 $1 - 2^h$ 个节点。

```
1 int countNodes(struct TreeNode* root){
2     if(root == NULL) {
3         return 0;           // (1)
4     }                       // (2)
5     return countNodes(root->left) + countNodes(root->right) + 1;
6 }
```

- (1) 空树的结点数为 0；
- (2) 非空树的结点数为 左子树个数 + 右子树个数 + 1；

4、开幕式

给定一棵二叉树 `root` 代表焰火，节点值表示巨型焰火这一位置的颜色种类。请计算巨型焰火有多少种不同的颜色。

```
1 int Hash[1024];
2
3 void transfer(struct TreeNode* root) {
4     if(root) {
5         Hash[root->val] = 1;      // (1)
6         transfer(root->left);    // (2)
7         transfer(root->right);   // (3)
8     }
9 }
10 int numColor(struct TreeNode* root){
11     int i, sum = 0;
12     memset(Hash, 0, sizeof(Hash));
13     transfer(root);
14     for(i = 1; i <= 1000; ++i) {
15         if(Hash[i]) ++sum;
16     }
17     return sum;
18 }
```



- (1) 将二叉树的当前结点标记；
- (2) 递归遍历左子树；
- (3) 递归遍历右子树；

三、推荐学习

♥ 《夜深人静写算法》 ♥

[\(一\) 搜索入门](#)

四、课后习题

课后所有 [必做] 习题，在上文都能找到答案，所以务必全部刷完。
坚持！加油！你可以的！

序号	题目链接	难度	必做
1	阶乘后的零	★☆☆☆☆	1
2	将数字变成 0 的操作次数	★☆☆☆☆	1
3	完全二叉树的节点个数	★☆☆☆☆	1
4	开幕式焰火	★☆☆☆☆	1
5	整数替换	★☆☆☆☆	0
6	二叉搜索树的范围	★★☆☆☆	0
7	二叉树的深度	★★☆☆☆	0
8	二叉树的最大深度	★★☆☆☆	0

序号	题目链接	难度	必做
9	翻转二叉树	★★☆☆☆	0
10	所有路径	★★☆☆☆	0
11	所有路径 II	★★☆☆☆	0

👉 关注公众号 观看 精彩学习视频 👈